AIN SHAMS UNIVERSITY
FACULTY OF ENGINEERING
i-CREDIT HOURS ENGINEERING PROGRAMS

MICROGRAD

Documentation Report

Assignment 1: Scalar Automatic Differentiation Engine

Course	CSE485–CSE488: Deep Learning
Department / Program	Computer Engineering and Artificial Intelligence Engineering
Student	Adel Waheed
Student ID	23P0213
Implementation branch	adel-waheed_23P0213

Prepared to document the concepts, mathematical foundations, design choices, and demonstrated results of the MicroGrad assignment.

Abstract

This report documents a scalar automatic differentiation engine and a small neural-network API inspired by MicroGrad. The assignment focuses on constructing computational graphs, applying the chain rule through reverse-mode automatic differentiation, ordering graph operations with a depth-first topological traversal, and building neural-network components that use the scalar engine. A demonstration notebook was used to exercise scalar arithmetic, gradients, activation functions, model construction, and a short training run. The report explains the mathematical basis and design rationale, and records the numerical results visible in that demonstration.

1. Introduction and Objectives

Deep-learning libraries automate differentiation, but their internal operation can be understood by implementing a small engine for scalar values. Each scalar stores its numerical value and the derivative of a final objective with respect to that value. Mathematical operations create graph nodes, and each node retains the local derivative rule needed during the backward pass.

The assignment objectives are:

- Represent scalar values and their gradients in a computational graph.
- Overload arithmetic operations such as addition, multiplication, and powers.
- Implement local backward rules for arithmetic operations and the tanh and exponential activation functions.
- Compute global gradients using reverse-mode backpropagation and topological sorting.
- Build Neuron, Layer, and MLP components with random initialization and forward passes.

2. Computational Graph and Value Representation

A computational graph is a directed acyclic graph (DAG). Leaf nodes represent input values or parameters; operation nodes represent calculations performed from earlier nodes. An edge from an input node to an operation node indicates that the operation depends on that input. The graph stores the structure of the calculation so that derivatives can be propagated after the forward computation.

A Value object conceptually contains:

- **data:** the scalar numerical value produced during the forward pass.
- **grad:** the accumulated derivative of the final output with respect to this value.
- **_prev:** references to the immediate predecessor values used to create the node.
- **_backward:** a local derivative operation that distributes the node's gradient to its predecessors.

For a scalar output L and an intermediate value v, the stored gradient represents ∂L/∂v. Gradients are accumulated because a value may contribute to the output through more than one path.

3. Mathematical Derivations

3.1 Addition

For z = a + b, the local derivatives are:

$$\partial z/\partial a = 1 \quad \partial z/\partial b = 1$$

If the upstream derivative is ∂L/∂z, the chain rule gives:

$$\partial L/\partial a = \partial L/\partial z \quad \partial L/\partial b = \partial L/\partial z$$

Thus, the backward rule adds the output gradient to the gradients of both operands.

3.2 Multiplication

For z = a × b, the local derivatives are:

$$\partial z/\partial a = b \quad \partial z/\partial b = a$$

Applying the chain rule:

$$\partial L/\partial a \mathrel{+}= b \times \partial L/\partial z \quad \partial L/\partial b \mathrel{+}= a \times \partial L/\partial z$$

The += notation emphasizes gradient accumulation. It is important when a node is reused or receives contributions from multiple downstream operations.

3.3 Power

For z = a■, where n is a constant exponent, the derivative with respect to the base is:

$$\partial z/\partial a = n \times a■■^{1}$$

The corresponding gradient contribution is (n × a■■¹) × ∂L/∂z. This is the standard power rule used for scalar powers.

3.4 Hyperbolic Tangent

For z = tanh(a), the derivative is:

$$\partial z/\partial a = 1 - \tanh^2(a) = 1 - z^2$$

Therefore, the input receives (1 – z²) × ∂L/∂z. Using the already computed output z avoids recomputing tanh during the backward pass.

3.5 Exponential

For z = exp(a) = e■, the derivative is equal to the function itself:

$$\partial z/\partial a = \exp(a) = z$$

The gradient contribution to a is z × ∂L/∂z.

3.6 Chain Rule

If z depends on an intermediate variable u and u depends on a, then:

$$\partial L/\partial a = (\partial L/\partial z) \times (\partial z/\partial u) \times (\partial u/\partial a)$$

Reverse-mode automatic differentiation applies this principle repeatedly, starting at the output and propagating derivatives toward the leaves.

4. Reverse-Mode Backpropagation and Topological Sorting

The output node is assigned an initial gradient of 1 because ∂L/∂L = 1. The engine then visits nodes in reverse topological order. A node's local backward rule uses its already-computed gradient to contribute derivatives to its predecessors.

4.1 Building the Topological Order

The assignment specifies a depth-first search (DFS) traversal. The traversal maintains an empty list called topo and a visited set. For each node, it first marks the node as visited, recursively visits every predecessor, and appends the node only after all predecessors have been visited. This is post-order DFS.

4.2 Why Post-Order and a Visited Set Matter

- **Post-order:** predecessors appear before the operation that depends on them in the resulting list. Reversing the list therefore places downstream operations before their inputs during backpropagation.
- **Visited set:** prevents revisiting a node in a graph with shared subexpressions. It avoids redundant traversal and protects against repeated processing of the same graph node.
- **Gradient accumulation:** if a value is used more than once, every valid path contributes to its gradient. The visited set controls graph traversal; it must not suppress legitimate gradient contributions from distinct operations.

4.3 Backward Pass Procedure

- Build the topological list from the output node using DFS.
- Set the output node's gradient to 1.0.
- Iterate over the topological list in reverse order.
- Call each node's local backward function to accumulate gradients in its predecessors.

4.4 Worked Computational-Graph Example

The demonstration notebook evaluates a = 2, b = 3, and c = a × b + a. The forward calculation is:

$$a \times b = 2 \times 3 = 6 \quad c = 6 + 2 = 8$$

The derivatives are:

$$\partial c/\partial a = b + 1 = 3 + 1 = 4 \quad \partial c/\partial b = a = 2$$

The notebook output reports c = 8, dc/da = 4, and dc/db = 2. These values agree with the analytical derivatives and provide a direct check of gradient propagation through a graph in which a is used twice.

5. Neural Network API Design

The assignment asks for three building blocks: Neuron, Layer, and MLP. They provide a hierarchy that turns scalar operations into a feed-forward neural network.

5.1 Neuron

A neuron receives an input vector x = (x■, …, x■), has weights w = (w■, …, w■), and a bias b. Its weighted sum is:

$$s = \Sigma■\ w■x■ + b$$

With a tanh activation, its output is:

$$y = \tanh(s)$$

The weights and bias are trainable parameters. Random initialization breaks symmetry between neurons; the bias allows the activation to shift independently of the weighted input.

5.2 Layer

A layer groups multiple neurons that receive the same input vector. Each neuron produces one output. For a layer with m neurons, the layer output is the collection (y■, …, y■). A layer's parameter collection is the concatenation of the parameters of its neurons.

5.3 Multi-Layer Perceptron (MLP)

An MLP composes layers sequentially. The output of one layer becomes the input of the next. For layer functions f■, f■, …, f■, the complete model is:

```
■ = f■(f■■■(… f■(f■(x)) …))
```

This design keeps the implementation modular: Neuron handles a single unit, Layer handles a group of units, and MLP handles the sequence of layers. Parameter collection can be performed recursively by combining the parameters of each contained neuron.

6. Demonstration Notebook and Observed Results

The notebook demonstrated the following operations and outputs. Values below are transcribed from the executed notebook screenshots shared during the assignment workflow.

Experiment	Observed result	Interpretation
Scalar graph	a=2, b=3, c=8; dc/da=4; dc/db=2	Forward arithmetic and accumulated gradients match the analytical result.
Exponential	Input=1.0; exp output≈2.7182818; gradient≈2.7182818	The derivative of exp(a) equals exp(a).
MLP forward pass	Architecture: 3 → 4 → 4 → 1; 41 parameters	The model accepts three inputs and produces one scalar output.
Training example	100 epochs; target=1.0; final prediction≈0.9502808	The prediction moved close to the target during the demonstration training run.
Final loss / error	Loss≈0.0024720; absolute error≈0.0497192	The final prediction is near, but not equal to, the target.

6.1 Interpretation of Training

The training plot shown in the notebook decreases over the displayed iterations. A decreasing loss indicates that the parameter updates reduced the chosen prediction error on this example. The final loss is approximately 0.002472, and the absolute prediction error is approximately 0.049719. These results demonstrate a successful small-scale optimization example; they do not, by themselves, establish generalization to unseen data.

7. Design Justifications

- **Scalar computation:** using scalar Value objects makes each operation and its derivative explicit, matching the educational goal of understanding automatic differentiation.
- **Local backward functions:** each operation owns its local derivative rule. This separates the mathematics of individual operations from the global graph traversal.

- **Reverse-mode differentiation:** the method is appropriate when a scalar loss depends on many inputs or parameters, because one backward traversal computes derivatives with respect to all graph leaves.
- **Topological DFS:** the post-order traversal guarantees a valid dependency ordering, while the visited set handles shared graph nodes efficiently.
- **Modular neural-network classes:** composing Neuron, Layer, and MLP keeps the implementation reusable and makes parameter collection and forward evaluation easier to organize.
- **Reproducibility:** the notebook sets Python and NumPy random seeds to 42, so random initialization and any NumPy-based random operations can be reproduced under the same environment and execution conditions.

8. Conceptual Questions and Answers

Q1. What is automatic differentiation?
Automatic differentiation evaluates derivatives by applying the chain rule to the sequence of elementary operations in a program. It is distinct from symbolic differentiation, which manipulates algebraic expressions, and numerical finite differences, which approximate derivatives using small perturbations.

Q2. What is the difference between a local derivative and an accumulated gradient?
A local derivative describes the effect of one operation's output on one of its inputs. An accumulated gradient is the derivative of the final objective with respect to a node, including contributions from every path from that node to the output.

Q3. Why is the output gradient initialized to 1?
For the output L itself, the derivative $\partial L/\partial L$ is 1. This provides the starting value for the chain rule during reverse-mode differentiation.

Q4. Why is reverse topological order used?
A node must receive contributions from all operations that depend on it before its gradient is propagated further toward its own inputs. Reversing the post-order topological list processes downstream operations before their predecessors.

Q5. Why must gradients be accumulated?
A value can be used by multiple operations or appear along multiple paths. Its total derivative is the sum of the contributions from those paths, so a backward rule must add contributions rather than overwrite earlier ones.

Q6. What is the role of the activation function?
An activation function transforms a neuron's weighted sum. In this assignment, tanh is one of the specified activation functions. Its nonlinear behavior allows a composition of layers to represent nonlinear mappings.

Q7. What does gradient descent do?
Gradient descent updates parameters in the direction opposite to the loss gradient. For parameter θ, learning rate η, and loss L, the update is $\theta \leftarrow \theta - \eta(\partial L/\partial \theta)$.

Q8. What does a decreasing training loss show?
It indicates that the optimization procedure is reducing the measured objective on the training example or training data. It does not alone prove that the model will perform well on new data.

9. Conclusion

The MicroGrad assignment illustrates how a scalar computational graph can support reverse-mode automatic differentiation. The key mechanisms are local derivative rules, chain-rule gradient accumulation, and a post-order depth-first traversal reversed for backpropagation. The Neuron, Layer, and MLP abstractions build a small feed-forward network from these scalar operations. The demonstration notebook's scalar-gradient checks, exponential derivative, MLP forward pass, and decreasing training loss provide evidence that the implemented workflow can be exercised end to end.

10. References

[1] Ain Shams University, Faculty of Engineering, CSE485–CSE488: Deep Learning, “Assignment 1: MicroGrad (Scalar Autograd Engine),” supplied assignment handout, 3 pages.

[2] Andrej Karpathy, “micrograd,” GitHub repository. https://github.com/karpathy/micrograd (inspiration identified in the assignment handout).

[3] The student's executed MicroGrad demonstration notebook, screenshots and outputs shared during the assignment workflow (scalar graph, exponential derivative, MLP forward pass, and training demonstration).

Scope note: This report is based on the assignment handout and the notebook outputs visible in the conversation. The complete source files were not supplied for direct inspection; therefore, implementation-specific statements are framed at the level of the assignment design and observed notebook behavior. Please verify the report against your final code and add any instructor-specific conceptual questions if provided separately.